



**Universidad  
Nacional  
de San Martín**

UNIVERSIDAD NACIONAL DE SAN MARTÍN

EXAMEN PARCIAL DE PYTHON

PROGRAMACIÓN ORIENTADA A OBJETOS, DECORADORES Y REQUESTS

## Instrucciones

- Lee cuidadosamente cada consigna antes de programar.
- El examen tiene una duración de **2 horas**.
- Se evaluarán: funciones, comprensiones, uso de decoradores, requests, programación orientada a objetos y manejo de archivos y excepciones.
- Se descontarán puntos por errores de sintaxis, falta de claridad o no cumplir con el formato pedido de input y output.

# 1. Ejercicio 1 – Gestión de Cursos y Decoradores (50 pts)

Se desea implementar un pequeño sistema para gestionar cursos y estudiantes en una plataforma educativa.

1. (Lambda y comprensiones – 8 pts) A partir de la siguiente lista de diccionarios:

```
1 cursos = [  
2     {"nombre": "Python Inicial", "nivel": "bsico", "alumnos": 35},  
3     {"nombre": "Machine Learning", "nivel": "avanzado", "alumnos": 22},  
4     {"nombre": "Bases de Datos", "nivel": "intermedio", "alumnos": 28},  
5     {"nombre": "Programacin Web", "nivel": "bsico", "alumnos": 40},  
6 ]
```

Usa:

- Una lambda para filtrar solo los cursos de nivel "básico".
- Una **comprensión de listas** para obtener una lista con los nombres de esos cursos.

—

2. (Clases y POO – 12 pts) Implementa una clase `Curso` con:

- Atributos: `nombre`, `nivel`, `alumnos`.
- Un método `info()` que devuelva un string con el formato: "Curso: Python Inicial (Nivel: básico) - 35 alumnos".

Luego, genera una lista de instancias de `Curso` a partir de la lista original.

—

3. (Pilares de la POO – 10 pts) Extiende el sistema con las siguientes clases:

- `CursoBasico`: imprime un mensaje de bienvenida estándar.
- `CursoIntermedio`: además permite tareas prácticas.
- `CursoAvanzado`: incluye acceso a proyectos especiales.

Requisitos:

- Usa **herencia** para no repetir código.
- Sobrescribe un método `beneficios()` en cada clase para mostrar distintos mensajes (polimorfismo).

—

4. (Decoradores – 10 pts) Implementa un decorador `@verificar_cupos` que, antes de inscribir un alumno, verifique si el curso tiene menos de 40 alumnos. Si no hay cupo, debe lanzar una excepción personalizada `CupoLlenoError`.

—

5. (Archivos y Excepciones – 10 pts) Implementa una función `guardar_cursos(cursos)` que:

- Guarde en un archivo `cursos.txt` cada curso en una línea con el formato `nombre,nivel,alumnos`.
- Use manejo de excepciones para capturar errores de escritura y mostrar un mensaje claro en caso de fallo.

## 2. Ejercicio 2 – Generadores y API de Criptomonedas (50 pts)

Se desea construir un módulo para consultar precios de criptomonedas y generar reportes.

1. (**Requests – 10 pts**) Realiza una solicitud HTTP GET a la API pública de **CoinGecko** para obtener el precio actual en USD de Bitcoin, Ethereum y Solana. URL base:

`https://api.coingecko.com/api/v3/simple/price?ids=bitcoin,ethereum,solana&vs_currencies=usd`

Guarda la respuesta en un archivo `cripto.json`. Implementa manejo de excepciones para errores de conexión.

---

2. (**Archivos y Excepciones – 10 pts**) Carga el archivo `cripto.json` y procesa los precios obtenidos. Implementa control de excepciones para:

- Archivo no encontrado
  - Errores de decodificación JSON
- 

3. (**Generadores – 10 pts**) Implementa un **generador** llamado `filtrar_altas(precios, umbral)` que reciba el diccionario de precios y devuelva solo las criptomonedas con precio mayor al umbral dado.
- 

4. (**Lambda y comprensiones – 10 pts**) Usando:

- Una `lambda` para aplicar un aumento del 5 % a cada precio.
  - Una **comprensión de diccionarios** para construir un diccionario `cripto: nuevo_precio`.
- 

5. (**Clases + Decoradores – 10 pts**) Crea una clase `ReporteCripto` con:

- Atributos: `fecha`, `precios`.
- Un método `resumen()` que imprima la cripto más cara, la más barata y el promedio de precios.

Implementa un decorador `@titulo_bonito` que envuelva el método `resumen()` para mostrar un encabezado llamativo antes de imprimir.